

Restaurant Management System

1.Introduction

1.1 Purpose of Document

This document describes the requirements for developing a Smart Restaurant Management & Online Ordering System that supports core restaurant operations, including menu management, online ordering, order tracking, and role-based access for staff and customers.

It provides a clear guide for developers, stakeholders, and system designers to understand the system's objectives, functionalities, constraints, and required components.

The purpose of this document is to ensure that the system is developed according to the restaurant's operational needs, supports multi-role access, integrates with modern technologies such as Spring Boot, REST APIs, Docker, and microservices architecture , and provides a smooth experience for customers and restaurant staff.

1.2 Scope of Product

The Restaurant Management System will provide:

Customer Tools:

- Browsing the menu through categorized items.
- Adding items to the cart.
- Placing online orders (takeaway only for this version).
- Viewing and tracking order status (Pending → Preparing → Ready → Completed).
- Viewing past order history.

Staff Tools:

- Viewing newly submitted customer orders.
- Updating order status as the order progresses.

- Managing customer orders assigned to them.

Admin /manager Tools:

- Managing menu items (add/edit/remove).
- Assigning users to roles (Admin, Chef, Waiter, Customer).
- Viewing financial reports and daily statistics.
- Managing categories.
- Viewing all customer orders.
- Assigning roles to system users (Admin, Staff, Customer).

Notifications:

- Alerts for customers when their orders are ready.
- Notifications for staff when new orders arrive.

Technology Scope:

- Backend: Spring Boot + REST APIs + Microservices.
- Authentication/Authorization: Role-based access.
- Deployment: Docker containers.
- Database: SQL-based system for storing orders, users, payments, and menu items.

Boundaries:

- System does not include employee payroll or HR management.
- External integrations limited to payment methods only.
- System will not handle delivery logistics (drivers, external delivery apps).
- The system focuses on restaurant operations only (menu, orders, tables, staff roles).

1.3 Definitions, Acronyms, Abbreviations

- **RMS:** Restaurant Management System.
- **API:** Application Programming Interface.
- **REST:** Representational State Transfer (API architecture).

- **Admin:** User with full control over system configurations.
 - **Chef/Kitchen Staff:** Role responsible for preparing orders and update status.
 - **Customer:** Person ordering food or reserving tables.
-

1.4 References

- IEEE/ANSI 830-1993 Requirements Specification Standard.
 - Spring Boot Official Documentation.
 - Docker Official Documentation.
 - MySQL/PostgreSQL Documentation
-

1.5 Overview of Remainder of Document

- **Section 2: General Description** – Gives a complete overview of the system's purpose, major features, user types, and constraints.
 - **Section 3: Specific Requirements** – Provides detailed functional and non-functional requirements, including database, APIs, UI, and performance specifications.
 - **Section 4: Appendices** – Includes stakeholder details, collected data, and operational notes.
-

2. General Description

2.1 Product Perspective

The Smart Restaurant Management & Online Ordering System will act as a central platform for both customers and restaurant staff. It integrates digital ordering with restaurant operations, enabling customers to browse menus, place takeaway orders, and track order status in real time. Staff members can manage incoming orders, update statuses, and oversee daily restaurant operations. The system will be built using **Spring Boot**, with REST APIs, Dockerized deployment, microservices architecture, and a MySQL-based database, ensuring modularity, scalability, and ease of maintenance.

The system is intended to complement existing restaurant workflows without replacing the physical aspects of dining, such as table service, while providing efficiency and automation in order processing and management.

2.2 Product Functions

The system provides the following functionalities:

Customer Tools:

- Browse menu items categorized by type.
- Add items to the shopping cart.
- Place online takeaway orders.
- Track order status: Pending → Preparing → Ready → Completed.
- View past orders and order history.

Staff Tools:

- View newly submitted customer orders.
- Update order status as it progresses.

- Manage assigned customer orders efficiently.

Admin/Manager Tools:

- Manage categories of menu items
- Assign roles to system users(Admin ,chef ,customer).
- View all customer orders .
- View financial reports and daily statistics.

Notifications:

- Customers receive alerts when orders are ready
- Staff receives notifications when new orders are submitted.

Technology Scope:

- **Backend:** Spring Boot + REST APIs + Microservices.
- **Authentication/Authorization:** Role-based access control.
- **Deployment:** Docker containers.
- **Database:** SQL-based (MySQL/PostgreSQL) storing orders, users, payments, and menu items.

2.3 User Characteristics

- **Customers:** Users who browse the menu, place orders online, track their orders, and view order history. They need a responsive and easy-to-use interface.
- **Waiters/Kitchen Staff:** Users responsible for receiving, updating, and completing orders. They need real-time updates and notifications.

- **Admin/Manager:** Users who manage menus, categories, user roles, and track overall restaurant performance. They require dashboards and reporting tools for efficient decision-making.

2.4 General Constraints

Technical Constraints:

- Reliable internet connection is required.
- System should be compatible with major web browsers.
- Must ensure high availability for order management and notifications.
- Must comply with data privacy and security standards for users and payments.
- *Organizational Constraints:*
 - Prioritize essential features such as menu, orders, and role management.
 - Provide user support and guidance for customers and staff.

Regulatory Constraints:

- Payment integration must comply with financial regulations.
- Accessibility standards should be followed for users with disabilities.

2.5 Assumptions and Dependencies

Assumptions:

- Users have internet access to use the system.
- Payment gateways (Stripe, PayPal, etc.) remain operational.
- Staff will update order statuses promptly.

Dependencies:

- System relies on SQL database services.

- Frontend framework (React, Angular, or Vue) for smooth user interface.
- Payment gateway services for online order processing.

3. Specific Requirements

3.1 Functional Requirements

3.1.1 Customer Management

- The system shall allow customers to **register an account** with basic information (name, email, phone, password).
- Customers can **log in and log out** securely.
- Customers can **update their profile** (address, phone number).
- Role-based access ensures that only registered **customers** can place orders.

3.1.2 Menu Management

- Admins can **add, edit, or remove menu items** including name, description, price, and category.
- Admins can **manage categories** (e.g., appetizers, main course, beverages, desserts).
- Menu items are **visible to customers** with real-time availability.
- Customers can **browse menu by category** and **search items by name**.

3.1.3 Order Management

- Customers can add items to a cart and place an online takeaway order.

- Orders go through the following statuses: Pending → Preparing → Ready → Completed.
- Staff (Chef/Waiter) can view new orders and update the status accordingly.
- Admin can view all orders and generate reports of daily orders.
- Customers can track the status of their orders in real time.
- Customers can view past order history.

3.1.4 Payment Management

- The system shall support online payment options (integrated with payment gateways).
- Customers receive confirmation of successful payment.
- Admin and cashiers can view all transaction records.
- The system shall maintain a secure record of payments linked to customer orders.

3.1.5 Notifications

- Customers shall receive alerts when orders are ready.
 - Staff shall receive notifications for new incoming orders.
-

3.2 Non-Functional Requirements

3.2.1 Performance

- The system should handle **at least 100 concurrent users** without noticeable delay.
- Page load times for menu browsing and order placement should be **under 3 seconds**.
- Order status updates should **propagate in real-time** (<2 seconds delay).

3.2.2 Reliability & Availability

- The system should be available **24/7**, with **99.9% uptime**.
- Transactions and order updates must be **logged and recoverable** in case of system failure.
- Dockerized deployment ensures **easy recovery and scaling**.

3.2.3 Security

- All user data, including passwords, must be **encrypted**
- Role-based access control ensures that only authorized users can perform specific actions (Admin, Staff, Customer).
- All APIs must be **authenticated** and secured with JWT or OAuth2.
- Online payments must use **secure, PCI-compliant gateways**.

3.2.4 Maintainability

- Backend code must follow **Spring Boot best practices** for readability and modularity.
- Frontend and backend should be **loosely coupled** via REST APIs.
- Microservices architecture allows **independent updates** of modules like Order, Menu, and User services.

3.2.5 Portability

- The system should work on **modern web browsers** (Chrome, Firefox, Edge).
- Docker containers allow deployment across **different environments** without configuration issues.

3.2.6 Usability

- Customer-facing interface must be **intuitive** for quick menu browsing and order placement.

- Staff and Admin dashboards must provide **clear visual indicators** for order status, notifications, and statistics.
 - Mobile-responsive design to support **smartphone and tablet users**.
-

3.3 Interface Requirements

3.3.1 User Interface (UI) Requirements

Customer UI:

- Browse menu items by category, with images, descriptions, and prices.
- Add, remove, or update items in the shopping cart.
- Place takeaway orders online.
- Track order status in real-time (Pending → Preparing → Ready → Completed).
- View past order history.
- Receive notifications when order status changes.

Staff UI (Chef, Waiter):

- View newly submitted customer orders.
- Update order status as the order progresses.
- View only orders assigned to them.
- Receive notifications for new or updated orders.

Admin/Manager UI:

- Manage menu items (add/edit/remove).
- Manage categories of menu items.
- Assign roles to system users (Admin, Staff, Customer).
- View all customer orders and filter them.
- View financial reports and daily statistics.

Design:

- Simple, clean layout with clear buttons and menus.
- Responsive design for desktop, tablet, and mobile devices.

3.3.2 External Interface Requirements

Hardware Interface:

- System must be accessible via desktop, laptop, tablet, and smartphone.
- Server hardware must support Dockerized microservices deployment.

Software Interface:

- Backend: Spring Boot REST APIs for all operations.
- Frontend: Any modern framework/library (React, Angular, or HTML/CSS/JS).
- Database: SQL-based (MySQL or PostgreSQL).
- Notifications: WebSocket or polling for real-time updates.
- Payment integration: Compatible with external payment gateways like Fawry or PayPal.

3.3.3 Usability Requirements

- Interface should be intuitive and easy to navigate for all user roles.
- Minimal clicks required to perform key actions (place order, update status, view reports).
- Clear feedback messages for success, errors, or validations.
- Consistent design across all screens.
- Search and filter options for menu items and orders.

3.3.4 Accessibility Requirements

- All text must be readable (minimum 14px font size).
 - Color schemes should support color-blind users.
 - Screen-reader friendly labels for buttons, forms, and menus.
 - Keyboard navigation should be possible for all main actions.
 - Forms must have clear error messages for invalid inputs.
-

3.3.5 Security Interface Requirements

- Role-based access control enforced via backend APIs.
 - **Admin:** Full access to all features.
 - **Staff** (Chef, Waiter): Access limited to orders assigned to them.
 - **Customer:** Access limited to their own orders and profile.
- Authentication via JWT or OAuth2.
- Passwords must be stored securely (hashed + salted).
- HTTPS required for all data transmission.
- Input validation to prevent SQL injection, XSS, and other vulnerabilities.
- Secure API endpoints to prevent unauthorized access.

3.3.6 Logical Database Requirements

Database Tables:

- **Users:** id, name, email, role, password_hash, created_at, updated_at.
- **MenuItems:** id, name, category_id, price, image_url, description, available_stock.
- **Categories:** id, name.

- **Orders:** id, user_id, order_items (link to MenuItem), status, total_price, timestamp.
- **Payments:** id, order_id, amount, payment_method, timestamp.

Relationships:

- Users ↔ Orders: One-to-Many.
- Orders ↔ MenuItem: Many-to-Many.
- Categories ↔ MenuItem: One-to-Many.

Constraints:

- Primary keys for all tables.
- Foreign key constraints for relationships.
- Role-based access enforced in queries.

Performance:

- Optimize queries for order tracking and reporting.
- Index frequently queried columns (user_id, status, timestamp).







